Jacobian Descent For Multi-Objective Optimization

Valérian Rey — Simplex Lab
valerian.rey@gmail.com
Joint work with Pierre Quinton

February 11, 2026

GitHub
github.com/TorchJD/torchjd

TorchJD
torchjd.org

arXiv
arxiv.org/pdf/2406.16232

Table of contents

Theory
- Partial order
- Pareto optimality
- Jacobian descent
- Unconflicting projection of gradients (UPGrad)

Applications and experimentation
- Optimization trajectories
- Per-sample JD
- Multi-task learning

Conclusion

Theory

Partial order

Define the partial order $(\mathbb{R}^m, \preceq)$, for $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^m$, as

$$\begin{aligned} & \boldsymbol{u} \preceq \boldsymbol{v} & \\ \Leftrightarrow \qquad & u_i \leq v_i & \forall i \in [m] \end{aligned}$$

Partial order

Define the partial order $(\mathbb{R}^m, \preceq)$, for $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^m$, as

$$\begin{aligned} & \boldsymbol{u} \preceq \boldsymbol{v} & \\ \Leftrightarrow \quad & u_i \leq v_i & \forall i \in [m] \end{aligned}$$

Examples:

- $\begin{bmatrix} 2 \\ 1 \end{bmatrix} \preceq \begin{bmatrix} 2 \\ 2 \end{bmatrix}$

Partial order

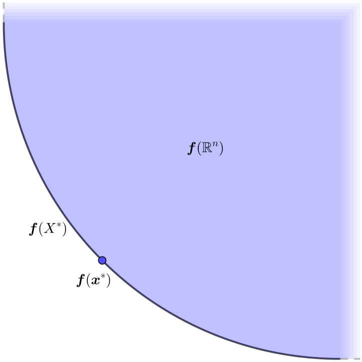
Define the partial order $(\mathbb{R}^m, \preceq)$, for $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^m$, as

$$\begin{aligned} & & \boldsymbol{u} \preceq \boldsymbol{v} & \\ \Leftrightarrow & & u_i \leq v_i & & \forall i \in [m] \end{aligned}$$

Examples:

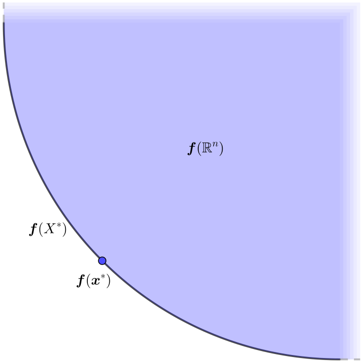
- $\begin{bmatrix} 2 \\ 1 \end{bmatrix} \preceq \begin{bmatrix} 2 \\ 2 \end{bmatrix}$
- $\begin{bmatrix} 2 \\ 1 \end{bmatrix}$ and $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$ are not comparable with this partial order.

Pareto optimality



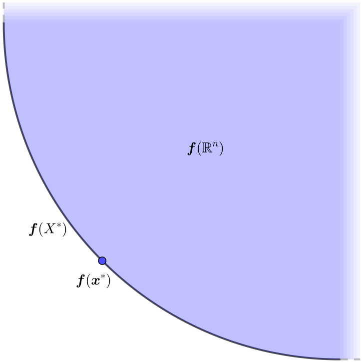
Pareto optimality

Pareto set: X^*



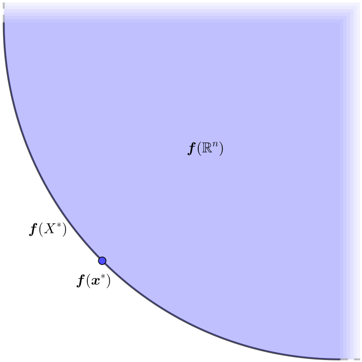
Pareto optimality

Pareto set: X^*
Pareto front: $\boldsymbol{f}(X^*)$



Pareto optimality

Pareto set: X^*
Pareto front: $\boldsymbol{f}(X^*)$



$$\boldsymbol{x}^* \in X^* \quad \Leftrightarrow \quad \forall \boldsymbol{y} \in \mathbb{R}^n,\ \boldsymbol{f}(\boldsymbol{y}) \preceq \boldsymbol{f}(\boldsymbol{x}^*) \rightarrow \boldsymbol{f}(\boldsymbol{y}) = \boldsymbol{f}(\boldsymbol{x}^*)$$

Jacobian descent

Goal: minimize $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$ according to the $\preceq$ partial order.

Jacobian descent

Goal: minimize $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$ according to the $\preceq$ partial order. For any $\boldsymbol{x} \in \mathbb{R}^n$, the Jacobian of $\boldsymbol{f}$ at $\boldsymbol{x}$, is

$$\mathcal{J}\boldsymbol{f}(\boldsymbol{x}) = \begin{bmatrix} \nabla f_1(\boldsymbol{x})^\top \\ \nabla f_2(\boldsymbol{x})^\top \\ \vdots \\ \nabla f_m(\boldsymbol{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_1(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\boldsymbol{x}) \\ \frac{\partial}{\partial x_1} f_2(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_2(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\boldsymbol{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_m(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\boldsymbol{x}) \end{bmatrix}$$

Jacobian descent

Goal: minimize $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$ according to the $\preceq$ partial order. For any $\boldsymbol{x} \in \mathbb{R}^n$, the Jacobian of $\boldsymbol{f}$ at $\boldsymbol{x}$, is

$$\mathcal{J}\boldsymbol{f}(\boldsymbol{x}) = \begin{bmatrix} \nabla f_1(\boldsymbol{x})^\top \\ \nabla f_2(\boldsymbol{x})^\top \\ \vdots \\ \nabla f_m(\boldsymbol{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_1(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\boldsymbol{x}) \\ \frac{\partial}{\partial x_1} f_2(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_2(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\boldsymbol{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_m(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\boldsymbol{x}) \end{bmatrix}$$

For a small enough update $\boldsymbol{y} \in \mathbb{R}^n$, Taylor's theorem states

$$\boldsymbol{f}(\boldsymbol{x} + \boldsymbol{y}) \approx \boldsymbol{f}(\boldsymbol{x}) + \mathcal{J}\boldsymbol{f}(\boldsymbol{x}) \cdot \boldsymbol{y}$$

Jacobian descent

Goal: minimize $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$ according to the $\preceq$ partial order. For any $\boldsymbol{x} \in \mathbb{R}^n$, the Jacobian of $\boldsymbol{f}$ at $\boldsymbol{x}$, is

$$\mathcal{J}\boldsymbol{f}(\boldsymbol{x}) = \begin{bmatrix} \nabla f_1(\boldsymbol{x})^\top \\ \nabla f_2(\boldsymbol{x})^\top \\ \vdots \\ \nabla f_m(\boldsymbol{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_1(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\boldsymbol{x}) \\ \frac{\partial}{\partial x_1} f_2(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_2(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\boldsymbol{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_m(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\boldsymbol{x}) \end{bmatrix}$$

For a small enough update $\boldsymbol{y} \in \mathbb{R}^n$, Taylor's theorem states

$$\boldsymbol{f}(\boldsymbol{x} + \boldsymbol{y}) \approx \boldsymbol{f}(\boldsymbol{x}) + \mathcal{J}\boldsymbol{f}(\boldsymbol{x}) \cdot \boldsymbol{y}$$

We want $\boldsymbol{f}(\boldsymbol{x} + \boldsymbol{y}) \preceq \boldsymbol{f}(\boldsymbol{x})$.

Jacobian descent

Goal: minimize $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$ according to the $\preceq$ partial order. For any $\boldsymbol{x} \in \mathbb{R}^n$, the Jacobian of $\boldsymbol{f}$ at $\boldsymbol{x}$, is

$$\mathcal{J}\boldsymbol{f}(\boldsymbol{x}) = \begin{bmatrix} \nabla f_1(\boldsymbol{x})^\top \\ \nabla f_2(\boldsymbol{x})^\top \\ \vdots \\ \nabla f_m(\boldsymbol{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_1(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\boldsymbol{x}) \\ \frac{\partial}{\partial x_1} f_2(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_2(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\boldsymbol{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_m(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\boldsymbol{x}) \end{bmatrix}$$

For a small enough update $\boldsymbol{y} \in \mathbb{R}^n$, Taylor's theorem states

$$\boldsymbol{f}(\boldsymbol{x} + \boldsymbol{y}) \approx \boldsymbol{f}(\boldsymbol{x}) + \mathcal{J}\boldsymbol{f}(\boldsymbol{x}) \cdot \boldsymbol{y}$$

We want $\boldsymbol{f}(\boldsymbol{x} + \boldsymbol{y}) \preceq \boldsymbol{f}(\boldsymbol{x})$. We select $\boldsymbol{y} = -\eta \mathcal{A}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x})) \in \mathbb{R}^n$.

Jacobian descent

The mapping $\mathcal{A} : \mathbb{R}^{m \times n} \to \mathbb{R}^n$ is called an **aggregator**, it parameterizes **Jacobian descent**:

Jacobian descent

The mapping $\mathcal{A} : \mathbb{R}^{m \times n} \to \mathbb{R}^n$ is called an **aggregator**, it parameterizes **Jacobian descent**:

Algorithm 1: Jacobian descent with aggregator $\mathcal{A}$

Input: $\boldsymbol{x} \in \mathbb{R}^n$, $0 < \eta$, $T \in \mathbb{N}$, $\mathcal{A} : \mathbb{R}^{m \times n} \to \mathbb{R}^n$

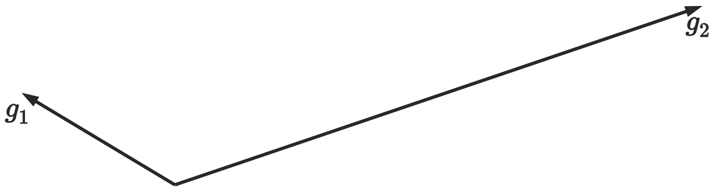
for $t \leftarrow 1$ **to** T **do**

$\quad \boldsymbol{x} \leftarrow \boldsymbol{x} - \eta \mathcal{A}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x}))$

Output: $\boldsymbol{x}$

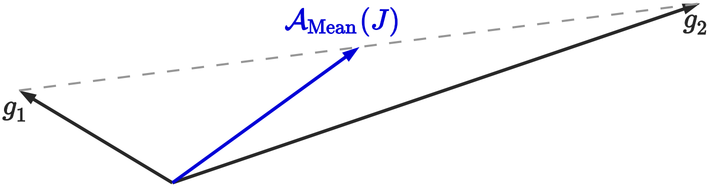
Unconflicting projection of gradients (UPGrad)

$$J = \begin{bmatrix} g_1^\top \\ g_2^\top \end{bmatrix}$$



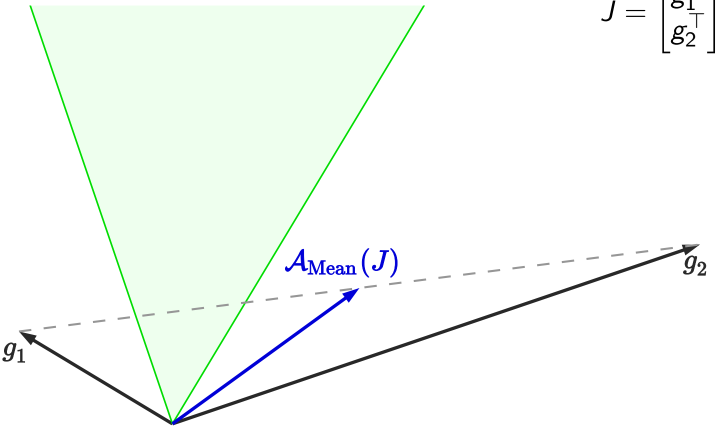
Unconflicting projection of gradients (UPGrad)

$$J = \begin{bmatrix} g_1^\top \\ g_2^\top \end{bmatrix}$$

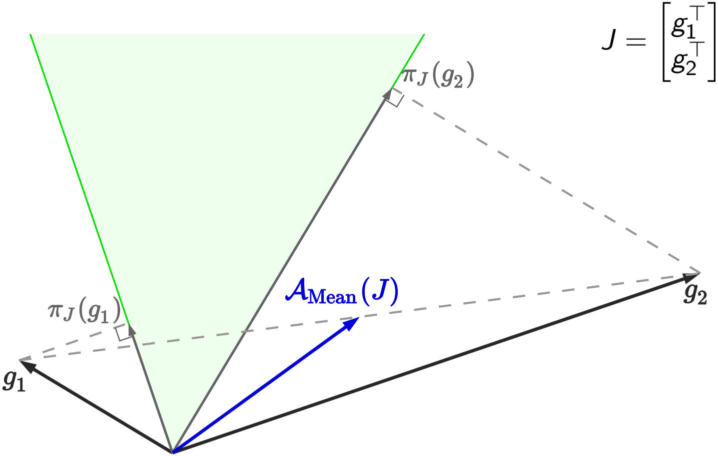


Unconflicting projection of gradients (UPGrad)

$$J = \begin{bmatrix} g_1^\top \\ g_2^\top \end{bmatrix}$$

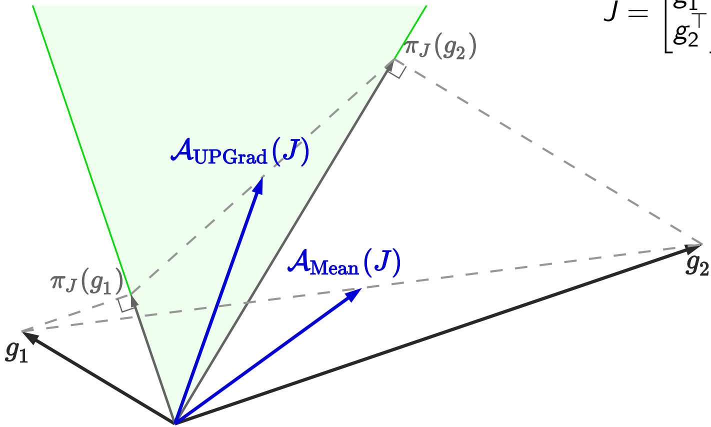


Unconflicting projection of gradients (UPGrad)



Unconflicting projection of gradients (UPGrad)

$$J = \begin{bmatrix} g_1^\top \\ g_2^\top \end{bmatrix}$$

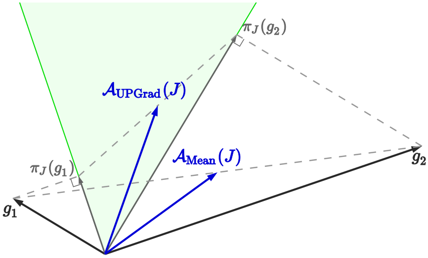


Formal definition of UPGrad

Given $J \in \mathbb{R}^{m \times n}$, let C_J be the cone $\{\boldsymbol{y} \in \mathbb{R}^n : \mathbf{0} \preceq J\boldsymbol{y}\}$.

$$\pi_J(\boldsymbol{x}) = \underset{\boldsymbol{y} \in C_J}{\arg\min} \|\boldsymbol{x} - \boldsymbol{y}\|^2$$

$$\mathcal{A}_{\text{UPGrad}}(J) = \frac{1}{m} \sum_{i \in [m]} \pi_J(g_i)$$

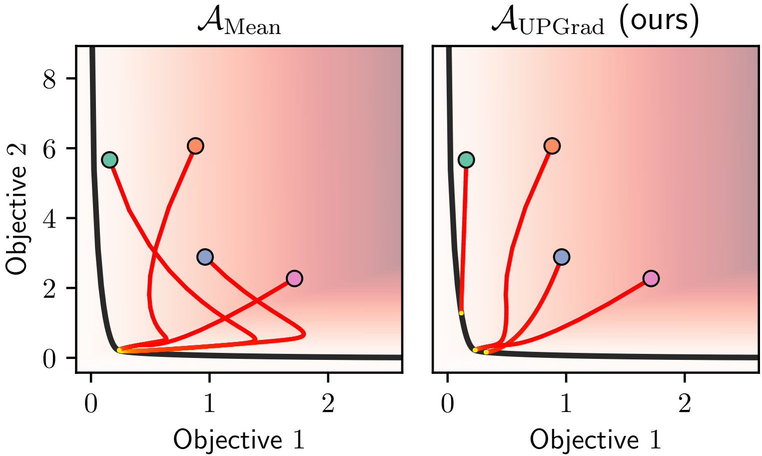


Applications and experimentation

Optimization trajectories

Goal: Minimize some convex quadratic function $\boldsymbol{f} : \mathbb{R}^2 \to \mathbb{R}^2$, with conflicting gradients.

Goal: Minimize some convex quadratic function $\boldsymbol{f} : \mathbb{R}^2 \to \mathbb{R}^2$, with conflicting gradients.



Optimization trajectories: code

```python
from torchjd import autojac
from torchjd.aggregation import UPGrad

x = tensor([0., 0.])  # initial value of x

for i in range(n_iters):
  y = f(x)  # shape: [2]
  autojac.backward(y)
  # x now has a .jac field
  autojac.jac_to_grad([x], UPGrad())
  # x now has a .grad field
  optimizer.step()
  optimizer.zero_grad()
```

Full project available at github.com/TorchJD/trajectories

Per-sample JD

Each element of the batch has its own loss.

Per-sample JD

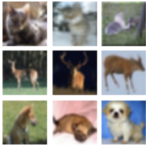
Each element of the batch has its own loss.

```
from torchjd import autojac
from torchjd.aggregation import UPGrad

batch_size = 16
model = Linear(10, 1)

for x, y in dataloader:
  z = model(x).squeeze(dim=1)  # shape: [16]
  losses = loss_fn(z, y)  # shape: [16]
  autojac.backward(losses)
  # parameters now have a .jac field
  autojac.jac_to_grad(model.parameters(), UPGrad())
  # parameters now have a .grad field
  optimizer.step()
  optimizer.zero_grad()
```

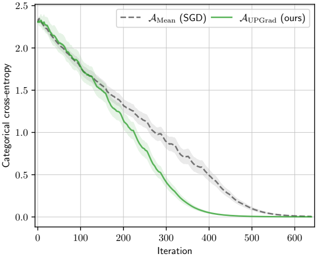
Full code example available at torchjd.org/latest/examples/iwrm.

Per-sample JD: results

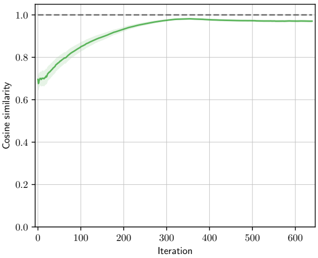
CIFAR-10



Training Loss



Update Similarity

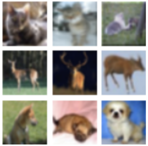


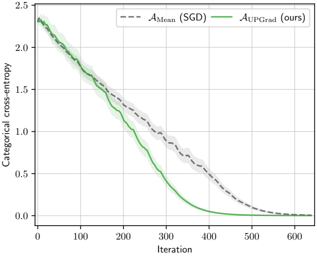
Per-sample JD: results

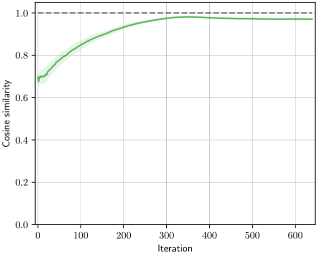
Training Loss

Update Similarity

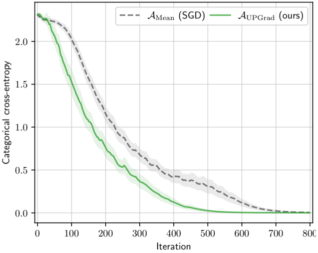
CIFAR-10

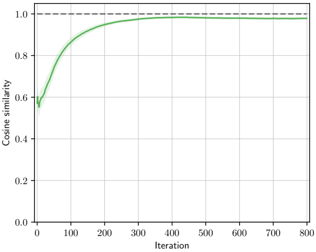






SVHN



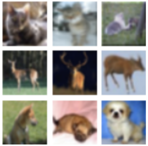


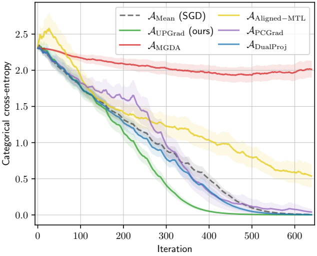
Per-sample JD: results

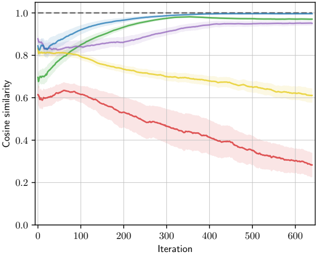
Training Loss

Update Similarity

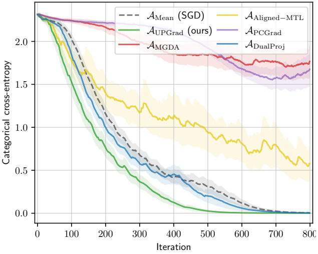
CIFAR-10

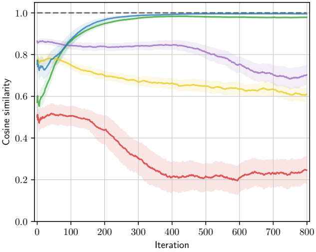






SVHN



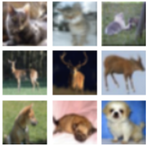


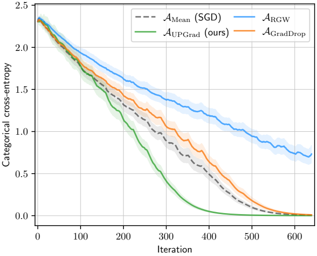
Per-sample JD: results

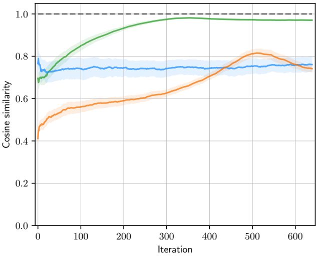
Training Loss

Update Similarity

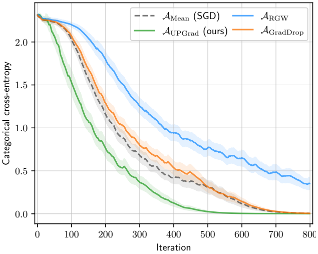
CIFAR-10

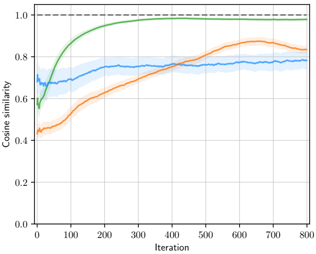






SVHN



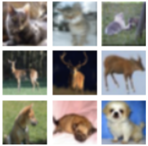


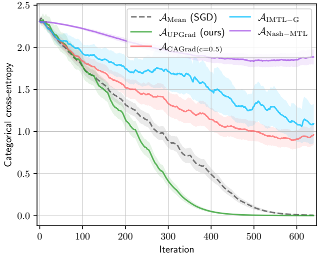
Per-sample JD: results

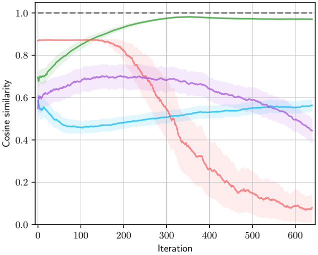
Training Loss

Update Similarity

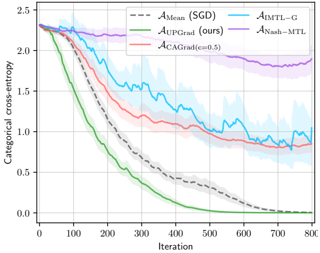
CIFAR-10

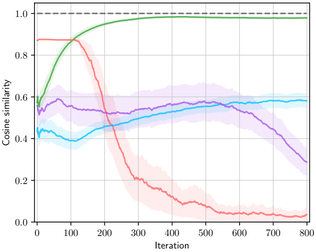






SVHN

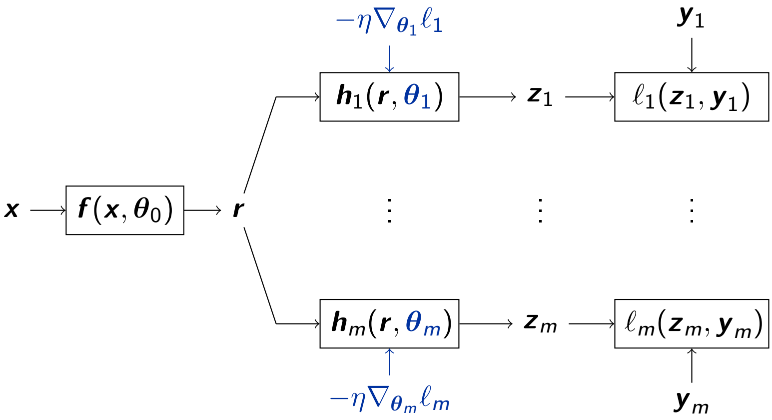




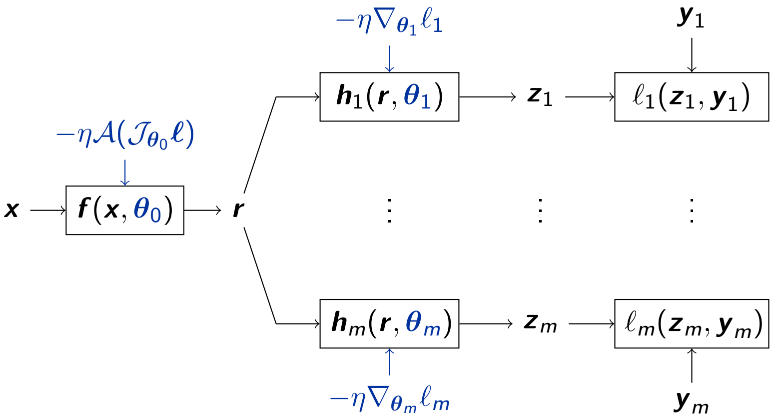
Multi-task learning

$$
\begin{tikzcd}
 & & & & & \boldsymbol{y}_1 \arrow[d] \\
 & & & \boxed{\boldsymbol{h}_1(\boldsymbol{r}, \boldsymbol{\theta}_1)} \arrow[r] & \boldsymbol{z}_1 \arrow[r] & \boxed{\ell_1(\boldsymbol{z}_1, \boldsymbol{y}_1)} \\
\boldsymbol{x} \arrow[r] & \boxed{\boldsymbol{f}(\boldsymbol{x}, \boldsymbol{\theta}_0)} \arrow[r] & \boldsymbol{r} \arrow[ur] \arrow[dr] & \vdots & \vdots & \vdots \\
 & & & \boxed{\boldsymbol{h}_m(\boldsymbol{r}, \boldsymbol{\theta}_m)} \arrow[r] & \boldsymbol{z}_m \arrow[r] & \boxed{\ell_m(\boldsymbol{z}_m, \boldsymbol{y}_m)} \\
 & & & & & \boldsymbol{y}_m \arrow[u]
\end{tikzcd}
$$

Multi-task learning



Multi-task learning



Multi-task learning: code

```
from torchjd import autojac
from torchjd.aggregation import UPGrad

f = Linear(10, 5)
h1 = Linear(5, 1)
h2 = Linear(5, 1)
for x, y1, y2 in dataloader:
  r = f(x)
  z1, z2 = h1(r), h2(r)
  l1, l2 = loss_fn(z1, y1), loss_fn(z2, y2)
  autojac.mtl_backward([l1, l2], features=r)
  # head parameters now have a .grad field
  # backbone parameters now have a .jac field
  autojac.jac_to_grad(f.parameters(), UPGrad())
  # All parameters now have a .grad field
  optimizer.step()
  optimizer.zero_grad()
```

Full code example available at torchjd.org/latest/examples/mtl.

Conclusion

Many problems are multi-objective.

Conclusion

Many problems are multi-objective.

Dual cone is rarely empty in deep learning.

Conclusion

Many problems are multi-objective.

Dual cone is rarely empty in deep learning.

TorchJD to compute and aggregate Jacobians.

Conclusion

Many problems are multi-objective.

Dual cone is rarely empty in deep learning.

TorchJD to compute and aggregate Jacobians.

Try to compute the inner products between your gradients.

Conclusion

Many problems are multi-objective.

Dual cone is rarely empty in deep learning.

TorchJD to compute and aggregate Jacobians.

Try to compute the inner products between your gradients.

Can JD solve problems that GD can't?

Appendix

Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x}))$ is a linear combination of the rows of $\mathcal{J}\boldsymbol{f}(\boldsymbol{x})$.

Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x}))$ is a linear combination of the rows of $\mathcal{J}\boldsymbol{f}(\boldsymbol{x})$.

The weights only depend on the (small) Gramian $\mathcal{G}\boldsymbol{f}(\boldsymbol{x}) = \mathcal{J}\boldsymbol{f}(\boldsymbol{x}) \cdot \mathcal{J}\boldsymbol{f}(\boldsymbol{x})^\top$.

Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x}))$ is a linear combination of the rows of $\mathcal{J}\boldsymbol{f}(\boldsymbol{x})$.

The weights only depend on the (small) Gramian $\mathcal{G}\boldsymbol{f}(\boldsymbol{x}) = \mathcal{J}\boldsymbol{f}(\boldsymbol{x}) \cdot \mathcal{J}\boldsymbol{f}(\boldsymbol{x})^\top$.

We can compute $\mathcal{G}\boldsymbol{f}(\boldsymbol{x})$ directly, extract the weights, combine the losses, and do a step of gradient descent.

Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x}))$ is a linear combination of the rows of $\mathcal{J}\boldsymbol{f}(\boldsymbol{x})$.

The weights only depend on the (small) Gramian $\mathcal{G}\boldsymbol{f}(\boldsymbol{x}) = \mathcal{J}\boldsymbol{f}(\boldsymbol{x}) \cdot \mathcal{J}\boldsymbol{f}(\boldsymbol{x})^\top$.

We can compute $\mathcal{G}\boldsymbol{f}(\boldsymbol{x})$ directly, extract the weights, combine the losses, and do a step of gradient descent.

We have developed the `autogram` engine to compute $\mathcal{G}\boldsymbol{f}(\boldsymbol{x})$ efficiently (work in progress).

Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x}))$ is a linear combination of the rows of $\mathcal{J}\boldsymbol{f}(\boldsymbol{x})$.

The weights only depend on the (small) Gramian $\mathcal{G}\boldsymbol{f}(\boldsymbol{x}) = \mathcal{J}\boldsymbol{f}(\boldsymbol{x}) \cdot \mathcal{J}\boldsymbol{f}(\boldsymbol{x})^\top$.

We can compute $\mathcal{G}\boldsymbol{f}(\boldsymbol{x})$ directly, extract the weights, combine the losses, and do a step of gradient descent.

We have developed the `autogram` engine to compute $\mathcal{G}\boldsymbol{f}(\boldsymbol{x})$ efficiently (work in progress).

This saves a lot of memory, so it's often much faster.

Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x}))$ is a linear combination of the rows of $\mathcal{J}\boldsymbol{f}(\boldsymbol{x})$.

The weights only depend on the (small) Gramian $\mathcal{G}\boldsymbol{f}(\boldsymbol{x}) = \mathcal{J}\boldsymbol{f}(\boldsymbol{x}) \cdot \mathcal{J}\boldsymbol{f}(\boldsymbol{x})^\top$.

We can compute $\mathcal{G}\boldsymbol{f}(\boldsymbol{x})$ directly, extract the weights, combine the losses, and do a step of gradient descent.

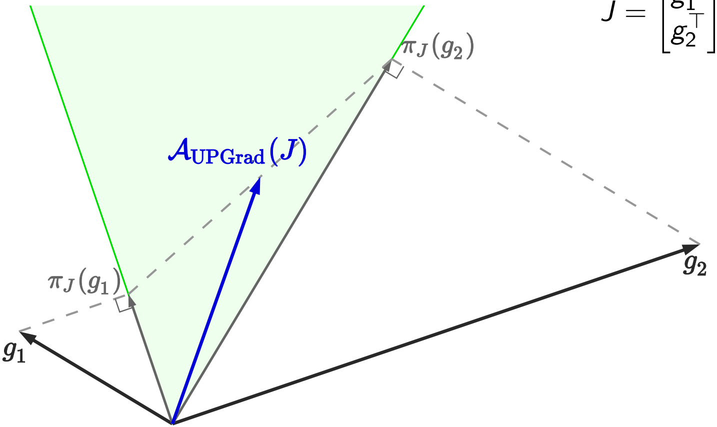
We have developed the **`autogram`** engine to compute $\mathcal{G}\boldsymbol{f}(\boldsymbol{x})$ efficiently (work in progress).

This saves a lot of memory, so it's often much faster.

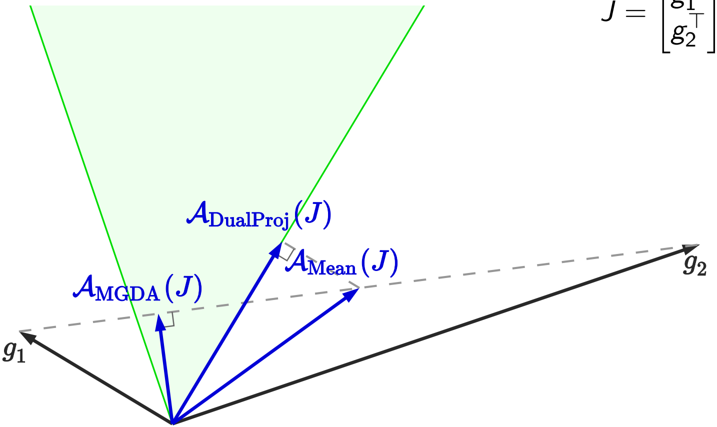
Applicable to most other aggregators from the literature.

Other aggregators

$$J = \begin{bmatrix} g_1^\top \\ g_2^\top \end{bmatrix}$$



Other aggregators

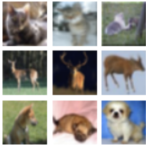


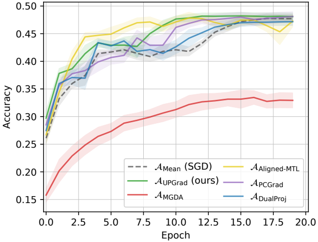
$$J = \begin{bmatrix} g_1^\top \\ g_2^\top \end{bmatrix}$$

Per-sample JD: test accuracies

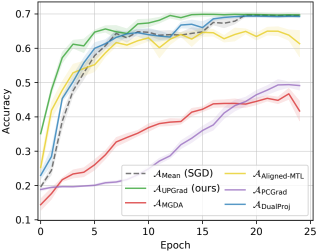
Test Accuracy

CIFAR-10





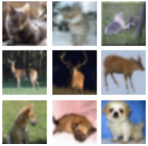
SVHN

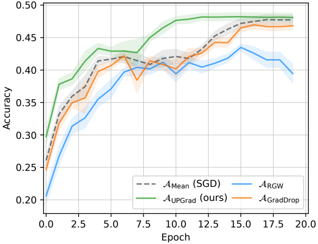


Per-sample JD: test accuracies

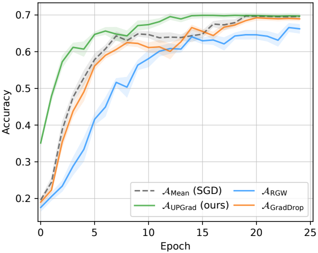
Test Accuracy

CIFAR-10





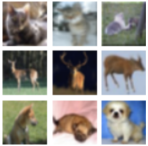
SVHN

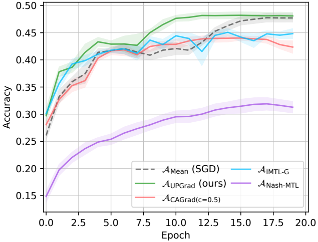


Per-sample JD: test accuracies

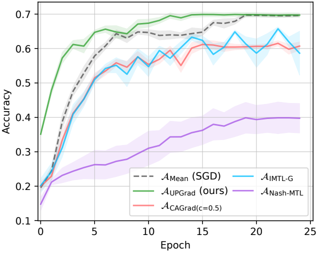
Test Accuracy

CIFAR-10



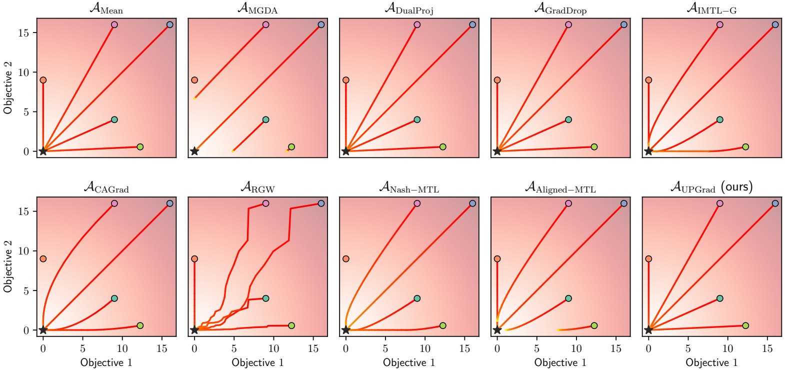


SVHN



Element-wise quadratic function trajectories

Minimize $\boldsymbol{f} : \begin{bmatrix} x \\ y \end{bmatrix} \mapsto \begin{bmatrix} x^2 \\ y^2 \end{bmatrix}$. It's Jacobian at $\begin{bmatrix} x \\ y \end{bmatrix}$ is $\begin{bmatrix} 2x & 0 \\ 0 & 2y \end{bmatrix}$.



Convex quadratic form trajectories

Some convex quadratic function $\boldsymbol{f} : \mathbb{R}^2 \to \mathbb{R}^2$, with conflicting gradients.